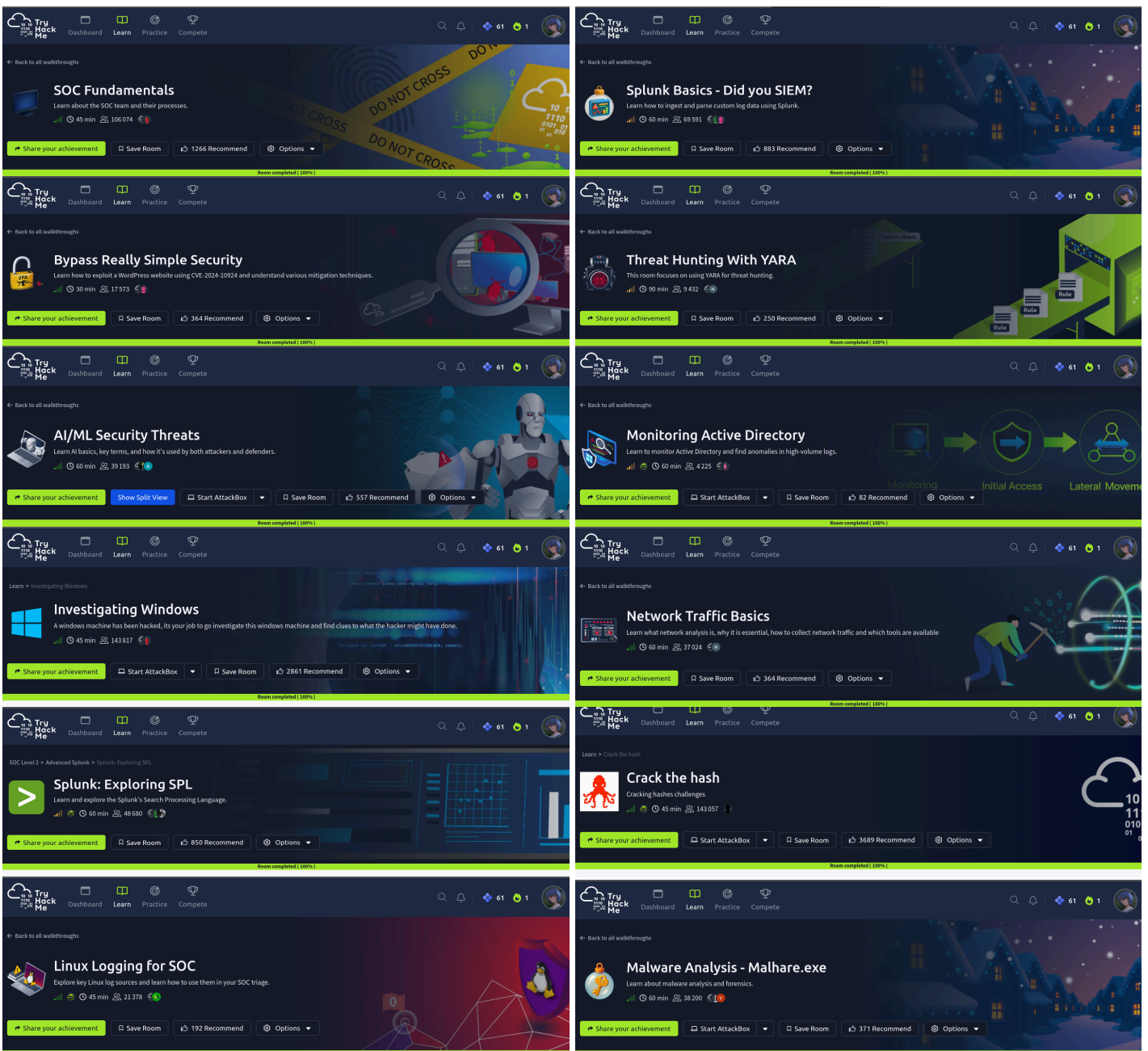


CS4230 – CyberCrime, IT Security and Risk

Project Assignments – Theme III

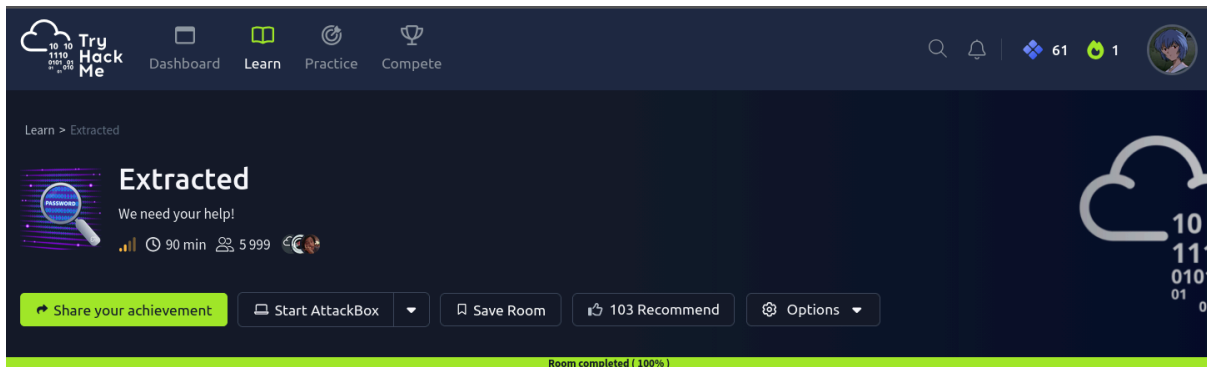
TryHackMe – SOC rooms screenshots



CS4230 – CyberCrime, IT Security and Risk

Project Assignments – Theme III

TryHackMe - Extracted Write-Up



1 Overview

In this room, we are working as a senior specialist and junior colleagues raised an alarm that some suspicious traffic was generated from one of the workstations, and nobody could figure out what was happening. Unfortunately, there was an issue with the ingesting the network traffic, but luckily, the network capture device was still working. So we are going to analyse the .pcapng file with Wireshark to see if we can find any interesting data. Key tools for this room are Wireshark, analysing PowerShell scripts, working with KeePassXC database, Hashcat and John-The-Ripper.

2 reconnaissance with Wireshark

Once we open Wireshark, it can be quite impressive and messy. Something we can do is look for different statistics, in order to see if there are strange things. The first thing I looked at was the conversation, endpoints and protocol hierarchy.

Wireshark · Conversations · traffic.pcapng

Ethernet · 2	IPv4 · 1	IPv6	TCP · 3	UDP															
Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A					
10.10.45.95	50356	10.10.94.106	1339	11	12 kB	0	5	462 bytes	6	12 kB	0.000000	0.0189	195 kbps	4 977 kbps					
10.10.45.95	50357	10.10.94.106	1337	53,314	390 MB	1	29,586	389 MB	23,728	1 MB	144.232932	3.2623	954 Mbps	3 191 kbps					
10.10.45.95	50358	10.10.94.106	1338	9	3 kB	2	5	3 kB	4	228 bytes	147.482076	0.0734	323 kbps	24 kbps					

Wireshark - Endpoints - traffic.pcapng									
Ethernet - 3	IPv4 - 2	IPv6	TCP - 6	UDP					
Address	Port	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes		
10.10.45.95	50356	11	12 kB	5	462 bytes	6	12 kB		
10.10.45.95	50357	53,314	390 MB	29,586	389 MB	23,728	1 MB		
10.10.45.95	50358	9	3 kB	5	3 kB	4	228 bytes		
10.10.94.106	1337	53,314	390 MB	23,728	1 MB	29,586	389 MB		
10.10.94.106	1338	9	3 kB	4	228 bytes	5	3 kB		
10.10.94.106	1339	11	12 kB	6	12 kB	5	462 bytes		

Wireshark - Protocol Hierarchy Statistics - traffic.pcapng									
Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	53338	100.0	390487858	21 M	0	0	0	53338
Ethernet	100.0	53338	0.2	746760	40 k	0	0	0	53338
Internet Protocol Version 4	100.0	53334	0.3	1066680	57 k	0	0	0	53334
Transmission Control Protocol	100.0	53334	0.3	1086728	58 k	23838	496808	26 k	53334
Sinec H1 Protocol	0.0	6	0.0	367	19	0	0	0	6
Hypertext Transfer Protocol	0.0	2	0.0	382	20	1	180	9	2
Data	0.0	1	0.0	1195	606	1	1195	606	1
Data	55.3	29494	99.1	387063394	20 M	29494	387063394	20 M	29494
Address Resolution Protocol	0.0	4	0.0	112	6	4	112	6	4

We can rapidly see that the traffic is almost filled with connections between 10.10.45.95 and 10.10.94.106 with data being sent. This is highly suspicious, so I decided to follow the TCP stream of the data.

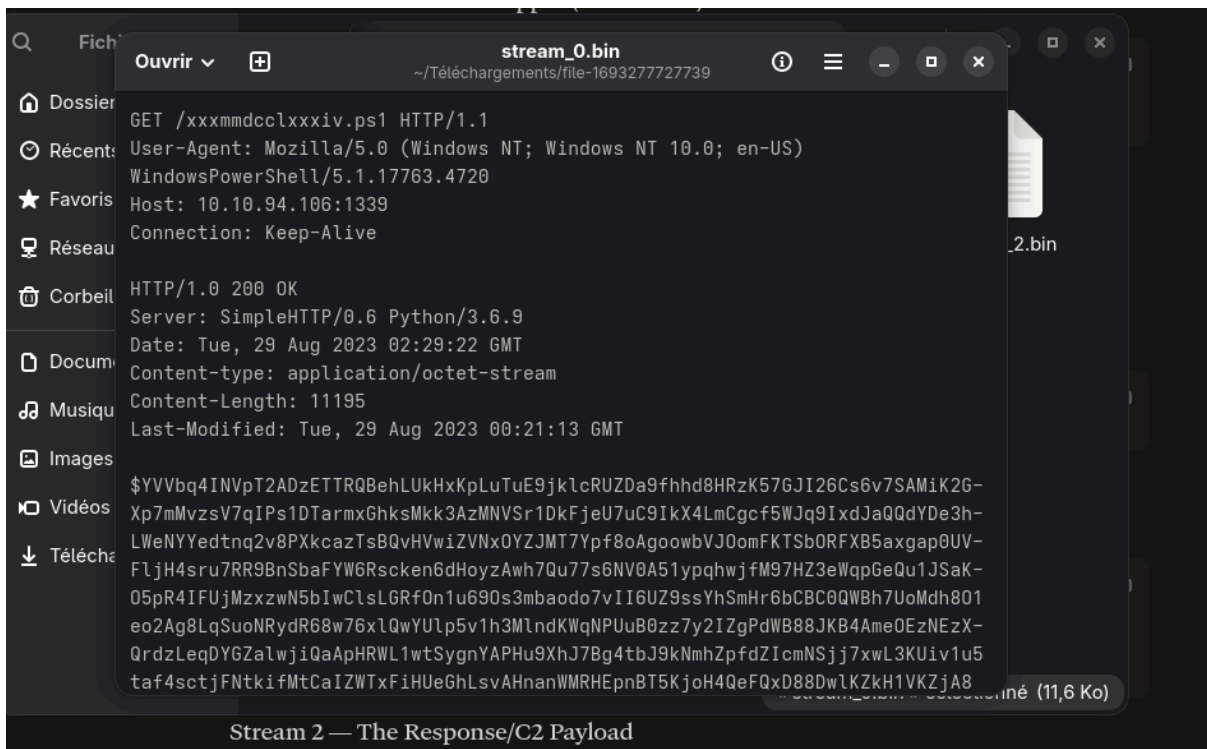
3 Extracting data stream

From there I will be on the Wireshark CLI because due to the size of the data sent, the GUI version is slow so the CLI version is more convenient.

```
> tshark -r traffic.pcapng -Y "ip.addr == 10.10.45.95 && ip.addr == 10.10.94.106" \
-T fields -e tcp.stream | sort -u
0
1
2
```

```
> for i in 0 1 2; do
  tshark -r traffic.pcapng -q -z follow,tcp,raw,$i \
    | tail -n +7 \
    | tr -d '\n\r' \
    | xxd -r -p > stream_${i}.bin
done
> file stream_0.bin stream_1.bin stream_2.bin
xxd stream_0.bin | head -4
xxd stream_1.bin | head -4
xxd stream_2.bin | head -4
stream_0.bin: ASCII text, with very long lines (1312), with CRLF, LF line terminators
stream_1.bin: ASCII text, with very long lines (65536), with no line terminators
stream_2.bin: ASCII text, with very long lines (2688), with no line terminators
00000000: 4745 5420 2f78 7878 6d6d 6463 636c 7878  GET /xxxxmddclxx
00000010: 7869 762e 7073 3120 4854 5450 2f31 2e31  xiv.ps1 HTTP/1.1
00000020: 0d0a 5573 6572 2d41 6765 6e74 3a20 4d6f  ..User-Agent: Mo
00000030: 7a69 6c6c 612f 352e 3020 2857 696e 646f  zilla/5.0 (Windo
00000040: 4441 554d 4564 4c6d 4975 4654 5155 4642  DAUMEdLmIuFTQUFB
00000050: 5955 4642 5155 4642 5155 4643 4761 776c  YUFBQUFBQUFCGawl
00000060: 5a31 6b48 5155 4642 5155 4643 5155 4642  Z1kHQUFBQUFCQUFB
00000070: 5655 4e42 5163 5648 5155 4651 5155 4642  VUNBQcVHQUFQQUFB
00000080: 515a 7667 3243 5735 4366 6444 516b 4643  QZvg2CW5CfdDQkFC
00000090: 5146 4a43 6334 4f77 7050 307a 4152 4c38  QFJCc40wpP0zARL8
00000100: 476b 646a 4b4c 3459 7655 4647 516b 4e43  GkdjKL4YvUFGQkNC
00000110: 516b 4a47 596b 4a37 5336 5757 3742 6b75  QkJGykJ7S6WW7Bku
```

We have 3 distinct streams that I extract and try to visualise, we can see that the stream 1 and 2 are not readable. However, the Stream 0 seems interesting, with a .ps1 script which is highly suspicious, let's dig into that.



4 Malware investigation

After further investigation and online research of the script's name, we can see that the PowerShell script "xxxmmdcclxxxiv.ps1" is known malware. This confirms that the PowerShell script has malicious behaviour, using ProcDump to steal process memory. In this case the malware tries to steal decrypted KeePass information from memory.

INTERACTIVE MALWARE ANALYSIS

[General](#)
[Behavior](#)
[MalConf](#)
[Static information](#)
[Video](#)
[Screenshots](#)
[System events](#)
[Network](#)

General Info

☒ Add for printing

File name:	xxxmmdcclxxxiv.ps1
Full analysis:	https://app.any.run/tasks/b6b839d8-3fb9-4b32-88bf-c877cec274ef
Verdict:	Malicious activity
Analysis date:	March 06, 2025 at 07:58:36
OS:	Windows 10 Professional (build: 19045, 64 bit)
Tags:	arch-exec arch-doc
Indicators:	
MIME:	text/plain
File info:	ASCII text, with very long lines (1312)
MD5:	9010749735DFAD6AFE2FD1FE2B84B9BF
SHA1:	EB73EC19A8C30545106BEB02731D1B709D689D61
SHA256:	E17F17898345EA047929E1E2BA042E2281DD816634183B6BB569F90AF0A60E5
SSDEEP:	192:Uj3pKTW+HNMjULpPh+J4Vn8YpKTW+INGfu4nZzFjFn5zVivn8YpKTW+INFpCs:UjvGW+LMjy1V8YGW+LGfuHGVVIV8YGW

ANY.RUN is an interactive service which provides full access to the guest system. Information in this report could be distorted by user actions and is provided for user acknowledgement as it is. ANY.RUN does not guarantee maliciousness or safety of the content.

Software environment set and analysis options

Reading the extracted script explains why we cannot read or interact with the stream 1 and 2, it's because it was obfuscated. The code revealed the full encoding logic with the stream, it corresponds to the KeePass dump XORed with 0x41 and encoded with Base64. And the Stream 2 corresponds to the .kdbx database XORed with 0x42 and also encoded with Base64. So we can now decode the two streams.

I used this code to extract the memory dump :

```
cat stream_1.bin | base64 -d > exfiltrated.bin
file exfiltrated.bin
xxd exfiltrated.bin | head -8
```

```
python3 -c "
data = open('exfiltrated.bin','rb').read()
open('lsass.dmp','wb').write(bytes([b ^ 0x41 for b in data]))
"
file lsass.dmp
```

```

mathias@archlinux:~/Téléchargements/file-1693277727739
~/Téléchargements/file-1693277727739

zsh: command not found: INFO:pypykatz:Parsing
zsh: ==== not found
> python3 -c "
from minidump.minidumpfile import MinidumpFile
mf = MinidumpFile.parse('lsass.dmp')
print('Modules found:')
for m in mf.modules.modules:
    print(' -', m.name)
"
Modules found:
- C:\Program Files\KeePass Password Safe 2\KeePass.exe
- C:\Windows\System32\ntdll.dll
- C:\Windows\System32\mscorEE.dll
- C:\Windows\System32\kernel32.dll
- C:\Windows\System32\KERNELBASE.dll
- C:\Windows\System32\advapi32.dll
- C:\Windows\System32\msvcrt.dll
- C:\Windows\System32\sechost.dll
- C:\Windows\System32\rpcrt4.dll
- C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorlib.dll
- C:\Windows\System32\shlwapi.dll
- C:\Windows\System32\combase.dll
- C:\Windows\System32\urlbase.dll
- C:\Windows\System32\bcryptPrimitives.dll
- C:\Windows\System32\gdi32.dll
- C:\Windows\System32\gdi32full.dll
- C:\Windows\System32\msvcp_win.dll
- C:\Windows\System32\user32.dll
- C:\Windows\System32\win32u.dll
- C:\Windows\System32\imm32.dll
- C:\Windows\System32\kernel.appcore.dll
- C:\Windows\System32\version.dll
- C:\Windows\Microsoft.NET\Framework64\v4.0.30319\clr.dll
- C:\Windows\System32\VC_RUNTIME140_CLR0400.dll
- C:\Windows\System32\urlbase_clr0400.dll
- C:\Windows\System32\psapi.dll
- C:\Windows\assembly\NativeImages_v4.0.30319_64\mscorlib\4bc5e5252873c08797895d5b6fe6ddfd\mscorlib.ni.dll
- C:\Windows\System32\ole32.dll
- C:\Windows\System32\ole32.dll

```

If we extract the dmp file from the .ps1 script we can see that this is confirmed by the keepass.exe dumped from memory.

5 Find the dumped masterkey

So now we are trying to find the dumped masterkey. I have used the tool of “matro7sh” from github “[keepass-dump-masterkey](#)”. This tool uses the CVE-2023-32784 exploit affecting certain

versions of KeePass. When a user types the master password, KeePass's .NET-based input box creates a managed string object in memory for each character and these strings are never securely wiped, even after the vault is locked or the process exits. This means the master password remains recoverable from any process memory dump. Once executed on the memory dump we can find our first password and respond to the first question.

```
> git clone https://github.com/matro7sh/keepass-dump-masterkey
cd keepass-dump-masterkey
```

```
> python3 poc.py ../lsass.dmp
2026-05-11 17:11:49,853 [.] [main] Opened ../lsass.dmp
Possible password: ●NoWaYIcanF0rGetThis123
```

What's the initial part of the password?

●NoWaYIcanF0rGetThis123

But we quickly notice that there is a strange character in the front of the password. After some research we can find that this is a placeholder because the exploit cannot recover the first character, meaning we need to find the missing character if we want to open the .kdbx file.

6 Unlock the vault

We need to extract the database from the Stream2, now that we know how the stream was encoded it is easy to decode it.

```
tshark -r traffic.pcapng -q -z follow,tcp,raw,2 \
| sed -n '/^[0-9a-f]/p' \
| tr -d ' \n\r' \
| xxd -r -p > stream2_clean.bin
```

```
base64 -d stream2_clean.bin > stream2_b64decoded.bin
```

```
python3 -c "
data = open('stream2_b64decoded.bin','rb').read()
open('Database1337.kdbx','wb').write(bytes([b ^ 0x42 for b in data]))
"
```

```
> tshark -r traffic.pcapng -q -z follow,tcp,raw,2 \
| sed -n '/^[0-9a-f]/p' \
| tr -d ' \n\r' \
| xxd -r -p > stream2_clean.bin
> base64 -d stream2_clean.bin > stream2_b64decoded.bin
> python3 -c "
data = open('stream2_b64decoded.bin','rb').read()
open('Database1337.kdbx','wb').write(bytes([b ^ 0x42 for b in data]))
"
> file Database1337.kdbx
xxd Database1337.kdbx | head -2
Database1337.kdbx: KeePass password database 2.x KDBX
00000000: 03d9 a29a 67fb 4bb5 0100 0300 0210 0031 ....g.K.....1
00000010: c1f2 e6bf 7143 50be 5805 216a fc5a ff03 ....qCP.X.!j.Z..
```

With that, we have the Database1337.kdbx file, the 03 D9 A2 9A in the beginning of the file confirms to us that it is a real KeePass database.

```
> ls
c2_response.bin    keepass-dump-masterkey  malware.txt    stream_1.bin    stream2_clean.bin
Database1337.kdbx  lsass.dmp              stream_0.bin    stream2_b64decoded.bin  stream2_raw_full.txt
exfiltrated.bin    malware.ps1            stream0_full.txt  stream_2.bin    traffic.pcapng
> keepass2john Database1337.kdbx > keepass.hash

> keepass2john Database1337.kdbx > keepass.hash
> cat keepass.hash
Database1337:$keepass$*2*60000*0*3909e7d4ae5b6c5a74e9f76e75b3e5879f2628fac7d5b4ab9e65bcc1999c6ad1*c23ecbbd379af0e1e680a1f90b8d51f7b018d3b053f00d06a9cdf2abc978037f*18c34bfb2b47c57ceedc0e0371db6240*ade7d7118bc18bfed4df7707a92043167c8fca1e532acdb569f51e01817db37a*2b47519844d1e750451d1a4814f1a31b089572d9b1cc6d7a2ce15a2a3bee5ca1
```

We convert the database into a hash that we can bruteforce with Hashcat , before using Hashcat we need to remove the “Database1337:” from the hash file. When it is done we can crack the password using this command line.

```
Hashcat -a 3 -m 13400 keepass_Hashcat .hash '?aNoWaYIcanF0rGetThis123'
```

Because we don’t know the first character, the ?a mask tells Hashcat to try all lowercase letters, uppercase letters, digits and symbols.

```
$keepass$*2*60000*0*3909e7d4ae5b6c5a74e9f76e75b3e5879f2628fac7d5b4ab9e65bcc1999c6ad1*c23ecbbd379af0e1e680a1f90b8d51f7b018d3b053f00d06a9cdf2abc978037f*18c34bfb2b47c57ceedc0e0371db6240*ade7d7118bc18bfed4df7707a92043167c8fca1e532acdb569f51e01817db37a*2b47519844d1e750451d1a4814f1a31b089572d9b1cc6d7a2ce15a2a3bee5ca1:?NoWaYIcanF0rGetThis123

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 13400 (KeePass (KDBX v2/v3))
Hash.Target.....: $keepass$*2*60000*0*3909e7d4ae5b6c5a74e9f76e75b3e58...ee5ca1
Time.Started.....: Mon May 11 17:49:31 2026 (3 mins, 20 secs)
Time.Estimated...: Mon May 11 17:52:51 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Mask.....: ?aNoWaYIcanF0rGetThis123 [23]
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 0 H/s (1.15ms) @ Accel:8 Loops:30 Thr:512 Vec:1
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 81/95 (85.26%)
Rejected.....: 0/81 (0.00%)
Restore.Point...: 0/1 (0.00%)
Restore.Sub.#01...: Salt:0 Amplifier:80-81 Iteration:59970-60000
Candidate.Engine.: Device Generator
Candidates.#01...: ?NoWaYIcanF0rGetThis123 -> ?NoWaYIcanF0rGetThis123
Hardware.Mon.#01.: N/A

Started: Mon May 11 17:49:14 2026
Stopped: Mon May 11 17:52:53 2026
```

We can see that the password is `?NoWaYIcanF0rGetThis123`, with the full password we can finally open the KeePass database. We stay in the CLI version using either the Python package `pykeepass` or the `keepassxc-CLI` tool to open the vault from the command line.

```
> python3 -c "
from pykeepass import PyKeePass

kp = PyKeePass('Database1337.kdbx', password='?NoWaYIcanF0rGetThis123')
print(f'[+] Vault unlocked! Found {len(kp.entries)} entries\n')
for entry in kp.entries:
    print(f'Title: {entry.title}')
    print(f'Username: {entry.username}')
    print(f'Password: {entry.password}')
    print(f'URL: {entry.url}')
    print('---')
"
[+] Vault unlocked! Found 3 entries

Title: Sample Entry
Username: User Name
Password: Password
URL: https://keepass.info/
---
Title: Sample Entry #2
Username: Michael321
Password: 12345
URL: https://keepass.info/help/kb/testform.html
---
Title: You win!
Username: None
Password: xWjy8SqH2CDDw76ptmvP
URL: None
---
>

> keepassxc-cli export --format csv Database1337.kdbx
Saisir le mot de passe pour déverrouiller Database1337.kdbx :
"Group","Title","Username","Password","URL","Notes","TOTP","Icon","Last Modified","Created"
"Database1337","Sample Entry","User Name","Password","https://keepass.info/","Notes","","0","2023-08-22T20:31:14Z","2023-08-22T20:31:14Z"
"Database1337","Sample Entry #2","Michael321","12345","https://keepass.info/help/kb/testform.html","","0","2023-08-22T20:31:14Z","2023-08-22T20:31:14Z"
"Database1337","You win!","","xWjy8SqH2CDDw76ptmvP","","THM{B3tt3r_Upd4t3_Y0ur_K33p455}","","0","2023-08-22T20:32:19Z","2023-08-22T20:31:24Z"
```

With that, we can find various stored passwords and our final flag.

What's the flag ?

THM{B3tt3r_Upd4t3_Y0ur_K33p455}

7 Conclusion and reflection

In this room we saw that even using a password manager it's still possible to open it, by dumping the decrypted master password from memory and sending it with the whole database to the hacker. We can ask ourselves if, inside a company, a password manager is relevant ?

First a password manager can help people to have a strong password by avoiding the need to memorise multiple credentials, they can use long, unique, randomly generated passwords for every system while needing to remember only 1 master password. Also, it can help to centralise all the

passwords in the company and can grant and revoke access when staff join or leave. With this centralisation it also helps the SOC to analyze who accessed with what password and when, which is very valuable during incident response.

But because of this strong advantage, it's also turning password managers into high-value targets. Because having the password vault means having access to every account and only one password is required, and as we demonstrated in this write-up, even if the master password is good (a password “?NoWaYIcanF0rGetThis123“ would take 435 years to be cracked according to [password strength meter](#)). However, flaws within the password manager itself can allow attackers to dump the password in clear from the memory and have access to the full database.

That said, a password manager is highly recommended inside a company. MFA can be required to access the vault, in our case, if MFA had been required, the attacker could not have accessed the vault. So a local password manager without an MFA in a corporate environment is dangerous. But an enterprise-grade solution with MFA enforced, zero-knowledge encryption, and centralized audit logging can really improve the security inside the organisation.